Unit-2 - Question 1

Operations of List in Python

Question 1: List and explain the operations of list with suitable examples.

① **Introduction:** Lists are ordered, mutable, allow duplicate elements, can store mixed data types.

② **Creating a list with examples:**

- `nums = [10, 20, 30]`
- `mixed = [1, "Python", 3.5]`
- `empty = []`

③ **Accessing elements:** indexing and negative indexing examples.

- `nums[0]` → 10
- `nums[2]` → 30
- `nums[-1]` → 30
- `nums[-2]` → 20

④ **Slicing:** list `[start : stop : step]`

- `nums[0:2]` → `[10, 20]` (start inclusive, stop exclusive)
- `nums[::2]` → `[10, 30]` (step of 2)

⑤ **Updating elements:**

- `nums[1] = 99` # nums becomes `[10, 99, 30]`

⑥ **Adding elements:**

- `append()` → `nums.append(40)` # `[10, 99, 30, 40]`
- `insert()` → `nums.insert(1, 55)` # `[10, 55, 99, 30, 40]`
- `extend()` → `nums.extend([50, 60])` # `[10, 55, 99, 30, 40, 50, 60]`

⑦ **Removing elements:**

- `remove(x)` → `nums.remove(55)` # `[10, 99, 30, 40, 50, 60]`
- `pop()` → `nums.pop()` # removes last element (60) → `[10, 99, 30, 40, 50]`
- `pop(i)` → `nums.pop(1)` # removes element at index 1 (99) → `[10, 30, 40, 50]`
- `del` → `del nums[0]` # removes first element → `[30, 40, 50]`
- `clear()` → `nums.clear()` # `[]` (empty list)

⑧ **Important list operations / operators:**

- Concatenation (+): `a = [1,2]; b = [3,4]` → `a + b` → `[1, 2, 3, 4]`
- Repetition (*): `a * 3` → `[1, 2, 1, 2, 1, 2]`
- Membership: `2 in a` → True ; `5 not in a` → True
- Traversal (for loop): `for x in a: print(x)` # prints 1 2

⑨ **Useful built-in functions:**

- `len(nums)` → 4
- `min(nums)` → 10
- `max(nums)` → 99
- `sum(nums)` → 199
- `sorted(nums)` → `[10, 30, 40, 50, 99]`

⑩ **Useful list methods:**

- `count(x)`: `nums.count(30)` → 1
- `index(x)`: `nums.index(40)` → 2 (first occurrence index)
- `sort()`: `nums.sort()` → `[10, 30, 40, 50, 99]` (in-place ascending)
- `reverse()`: `nums.reverse()` → `[99, 50, 40, 30, 10]` (in-place)
- `copy()`: `nums.copy()` → `[99, 50, 40, 30, 10]` (new list)

⑪ **Nested list example:**

- `matrix = [[1, 2], [3, 4]]`
  - `matrix[0]` → `[1, 2]`
  - `matrix[0][1]` → 2

⑫ **Conclusion / Summary:**

- lists are flexible and widely used for storing and processing collections of data.

Unit-2 – Question 2

Operations of Tuple in Python

Question 2: List and explain the operations of tuple with suitable examples.

① **Introduction:** tuple is ordered, immutable, allows duplicates.

② **Creating tuples:**
- t1 = (10, 20, 30)
- single element tuple: t2 = (5,) # comma is required
- empty tuple: t3 = ()

③ **Accessing elements:** indexing and negative indexing examples.
- t1 = (10, 20, 30)
- t1[0] → 10 t1[1] → 20 t1[2] → 30
- t1[-1] → 30 t1[-2] → 20 t1[-3] → 10

④ **Slicing with examples.**
- t1 = (10, 20, 30, 40, 50)
- t1[1:4] → (20, 30, 40)
- t1[2:] → (30, 40, 50)
- t1[::2] → (10, 30, 50)
- t1[:3] → (10, 20, 30)
- t1[:2] → (10, 30, 50)
- t1[::-1] → (50, 40, 30, 20, 10)

⑤ **Tuple packing and unpacking.**
- t1 = (10, 20, 30)
- a, b, c = t1 # unpacking
- a → 10, b → 20, c → 30

⑥ **Operations on tuples.**
- Concatenation (+): t1 = (10, 20), t2 = (30, 40) t1 + t2 → (10, 20, 30, 40)
- Repetition (*): t1 * 3 → (10, 20, 10, 20, 10, 20)
- Membership (in / not in):
  - 20 in t1 → True
  - 99 not in t1 → True
- Traversal (for loop):
  - for x in t1: print(x) # prints 10 20 30

⑦ **Built-in functions:**
- len(t1) → 3
- min(t1) → 10
- max(t1) → 30
- sum(t1) → 60

⑧ **Tuple methods.**
- count(x): counts occurrences of x in tuple.
  - t1 = (10, 20, 20, 30) t1.count(20) → 2
- index(x): returns first index of x.
  - t1.index(20) → 1 t1.index(30) → 3

⑨ **Conversion:**
- list(t1) → [10, 20, 30]
- tuple([1, 2, 3]) → (1, 2, 3)

⑩ **Nested tuples.**
- t = (1, (2, 3), (4, 5, 6))
  - t[1] → (2, 3)
  - t[1][0] → 2
  - t[2][2] → 6

⑪ **Important note on immutability.**
- Tuple elements cannot be changed directly.
- t1 = (10, 20, 30)
- t1[0] = 100 # TypeError: 'tuple' object does not support item assigmment

⑫ **Conclusion / Summary:**
- Tuples support various operations like indexing, slicing, concatenation, repetition, membership testing and built-in functions. They are immutable and widely used for fixed collections of data.

Unit-2 – Question 3

Operations of String in Python

Question 3 : List and explain the operations of string with suitable examples.

① **Introduction :** strings are ordered sequences of characters and are immutable.

② **Creating strings using** single quotes, double quotes, triple quotes.

- `s1 = 'Python'`
- `s2 = "Python"`
- `s3 = '''This is n multiple line \n string'''`

③ **Indexing and negative indexing** examples using `s = "Python"`.

- `s = "Python"`
- Index: `s[0]` → 'P' `s[1]` → 'y' `s[5]` → 'n'
- Negative index: `s[-1]` → 'n' `s[-2]` → 'o' `s[-6]` → 'P'

④ **Slicing examples.**

- `s = "Python"`
- `s[0:3]` → 'Pyt'
- `s[::-1]` → 'nohtyP' (reverse of string)

⑤ **String operators.**

- Concatenation (+): `s1 = "Py"; s2 = "thon"; s1 + s2` → 'Python'
- Repetition (*): `s * 3` → 'PythonPythonPython'
- Membership (in / not in): 'Py' in s → True; 'java' not in s → True
- Comparison: 'abc' < 'abd' → True; 'abc' == 'abc' → True

⑥ **Important string methods with examples.**

- lower(): `s = "PyThOn"; s.lower()` → 'python'
- upper(): `s.upper()` → 'PYTHON'
- strip(): `t = " Python "; t.strip()` → 'Python'
- replace(): `s.replace('Py', 'Hi')` → 'Hython'
- split(): `u = "a,b,c"; u.split(',')` → ['a', 'b', 'c']
- join(): `'-'.join(['a', 'b', 'c'])` → 'a-b-c'
- find(): `s.find('th')` → 2 ; `s.find('z')` → -1
- count(): `s.count('t')` → 1
- startswith(): `s.startswith('Py')` → True
- endswith(): `s.endswith('on')` → True

⑦ **Traversing a string using for loop.**

```
s = "Python"
for ch in s:
    print(ch)     # Output: P y t h o n
```

⑧ **Formatting strings using f-strings or format().**

- `name = "Alice"; age = 20`
- `f"My name is {name} and I am {age} years old."` → "My name is Alice and I am 20 years old.'
- `"My name is {} and I am {} years old.".format(name, age)` → same output

⑨ **Escape sequences with tiny examples.**

- `"Hello\nWorld"` →
  Hello
  World
- `"A\tB\tC"` → A B C

⑩ **Important note:** strings are immutable.

- We cannot change individual characters of a string after creation.
  Example: `s = "Python"; s[0] = 'p'` → TypeError

⑪ **Short conclusion / summary.**

- Python strings support a rich set of operations including indexing, slicing, operators, methods, traversal, formatting and escape sequences, making them powerful and easy to work with.

Unit-2 – Question 4

Operations of Dictionary in Python

Question 4: List and explain the operations of dictionary with suitable examples.

① **Introduction:** Dictionary stores data as key:value pairs, mutable, unordered mapping, keys are unique.

② **Creating dictionaries:**

- `student = {"name":"Riya", "age":20}`
- `empty = {}`

③ **Accessing values using key and get():**

- `student["name"]` → `'Riya'`
- `student.get("age")` → `20`
- `student.get("grade", "N/A")` → `'N/A'` # returns default if key not present

④ **Adding and updating items:**

- `student["grade"] = "A"` # adds new key
- `student["age"] = 21` # updates existing key

⑤ **Removing items using pop(), popitem(), del, clear():**

- `val = student.pop("grade")` # removes 'grade' and returns its value
- `item = student.popitem()` # removes and returns last inserted (key, value)
- `del student["age"]` # deletes key 'age'
- `student.clear()` # removes all items → {}

⑥ **Important methods:**

- `student.keys()` → `dict_keys(['name', 'age'])`
- `student.values()` → `dict_values(['Riya', 20])`
- `student.items()` → `dict_items([('name', "Riya'), ('age', 20)])`
- `student.update({"grade":"A", "age":21})` # add/update multiple items
- `student.copy()` → creates a shallow copy of dictionary

⑦ **Membership test in keys using in:**

- `"name" in student` → True
- `"grade" in student` → False

⑧ **Traversal using for key in d and for k,v in d.items():**

-
```
for key in student:
    print(key, "→", student[key])
```
-
```
for k, v in student.items():
    print(k, "→", v)
```

⑨ **Built-in functions:**

- `len(student)` → number of key:value pairs
- `sorted(student)` → returns sorted list of keys
  e.g., `sorted(student)` → `['age', 'name']`

⑩ **Nested dictionary example:**

- `marks = {"maths": {"score":88, "grade":"A"}, "science": {"score":76, "grade":"B"}}`
- Access: `marks["maths"]["score"]` → 88

⑪ **Real-life use:**

- Dictionaries are useful for storing structured records like student details, employee info, configuration settings, API responses, etc.

⑫ **Conclusion / Summary:**

- Dictionaries provide fast lookup by key, flexible updates, and many methods for efficient data handling.

Unit-2 - Question 5

Operations of Set in Python

Question 5: List and explain the operations of set with suitable examples.

① **Introduction:** sets are unordered collections, mutable, do not allow duplicate elements.

② **Creating sets:**
- `s = {1, 2, 3}`
- empty set using `set()`: `s = set()` # set() creates an empty set

③ **Adding elements using add() and update():**
- `s = {1, 2}`
- `s.add(3)` # adds a single element
  Output: `{1, 2, 3}`
- `s.update([4, 5])` # adds multiple elements from an iterable
  Output: `{1, 2, 3, 4, 5}`

④ **Removing elements using remove(), discard(), pop(), clear():**
- `s.remove(2)` # removes 2, error if not present
  Output: `{1, 3, 4, 5}`
- `s.discard(10)` # no error if 10 not present
  Output: `{1, 3, 4, 5}`
- `popped = s.pop()` # removes and returns a random element
  Output: `popped = 1` `s = {3, 4, 5}` (element may vary)
- `s.clear()` # removes all elements
  Output: `set()`

⑤ **Set operations with examples:**
- Union (| or union()):
  `A = {1,2,3}; B = {3,4,5}` `A | B → {1, 2, 3, 4, 5}` or `A.union(B) → {1, 2, 3, 4, 5}`
- Intersection (& or intersection()):
  `A & B → {3}` or `A.intersection(B) → {3}`
- Difference (-):
  `A - B → {1, 2}` (elements in A not in B)
  `B - A → {4, 5}` (elements in B not in A)
- Symmetric Difference (^):
  `A ^ B → {1, 2, 4, 5}` (elements in either A or B, not in both)

⑥ **Membership test using in / not in:**
- `3 in A → True`
- `10 not in A → True`

⑦ **Subset and Superset using <= and >= or issubset() / issuperset():**
- `C = {1, 2}; A = {1, 2, 3}`
- `C <= A → True` or `C.issubset(A) → True`
- `A >= C → True` or `A.issuperset(C) → True`

⑧ **Useful methods:**
- `s = {1, 2, 3}`
- `t = s.copy()` # creates a shallow copy
  Output: `t → {1, 2, 3}`

⑨ **Converting list to set to remove duplicates:**
- `nums = [1, 2, 2, 3, 4, 4, 5]`
- `unique = set(nums)` → `{1, 2, 3, 4, 5}`

⑩ **Important note:**
- Set items are unique.
- Sets are unordered, so indexing and slicing are not supported.

⑪ **Optional note on frozenset:**
- frozenset is an immutable version of set and can be used as a key in dictionaries or added to other sets.

⑫ **Conclusion / Summary:**
- Set operations help in storing unique items, performing mathematical set operations, membership testing, and removing duplicates efficiently.

Unit-2 - Question 6

Different Types of Comprehensions

Question 6 : Explain different types of comprehensions with examples.

① **Introduction :** comprehensions provide a short and readable way to create collections.

② **General syntax idea :**

- expression for item in iterable
- optional if condition

③ **List comprehension with simple examples :**

- Squares :
```
nums = [1, 2, 3, 4, 5]
squares = [x*x for x in nums]  # [1, 4, 9, 16, 25]
```
- Even numbers :
```
evens = [x for x in nums if x % 2 == 0]
# [2, 4]
```

④ **List comprehension with condition :**

- Positive multiples of 3 from 1 to 15
```
multiples = [x for x in range(1, 16) if x % 3 == 0]
# [3, 6, 9, 12, 15]
```

⑤ **Nested list comprehension :**

- Matrix (2 x 3) filled with 0
```
matrix = [[0 for j in range(3)] for i in range(2)]
# [[0, 0, 0], [0, 0, 0]]
```
- Flatten a 2D list
```
flat = [x for row in matrix for x in row]  # [0, 0, 0, 0, 0, 0]
```

⑥ **Set comprehension :**

- Unique squares from a list
```
nums = [1, 2, 2, 3, 4]
squares_set = {x*x for x in nums}  # {1, 4, 9, 16}
```

⑦ **Dictionary comprehension :**

- Map numbers to squares
```
d = {x: x*x for x in range(1, 6)}
# {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

⑧ **Generator expression :**

- Generate squares without creating a list
```
gen = (x*x for x in range(5))  # <generator object at 0x...>
```
- Note : returns a generator (lazy evaluation).

⑨ **Advantages :**

- Concise : fewer lines of code.
- Readable : easy to understand intent.
- Often faster : implemented in optimized way.

⑩ **Comparison note :**

- Comprehension is shorter and more Pythonic.
- Normal loop is more flexible for complex logic and multiple statements.

⑪ **Conclusion / Summary :**

- Comprehensions help write clean, efficient, and expressive Python code for creating collections.

Unit-2 - Question 7

Important Programs

Question 7: Programs

Important Unit-2 programs with simple code.

① Sum of elements in a list.

```
nums = [10, 20, 30, 40]
s = 0
for x in nums:
    s += x
print(s)    # Output: 100
```

② Find largest element in a list.

```
nums = [10, 45, 7, 23, 89]
mx = nums[0]
for x in nums:
    if x > mx:
        mx = x
print(mx)    # Output: 89
```

③ Remove duplicates from a list.

```
nums = [1, 2, 2, 3, 4, 4, 5, 1]
unique = []
for x in nums:
    if x not in unique:
        unique.append(x)
print(unique)   # Output: [1, 2, 3, 4, 5]
```

④ Reverse a string.

```
s = "python"
rev = s[::-1]
print(rev)    # Output: nohtyp
```

⑤ Count vowels in a string.

```
s = "programming"
vowels = "aeiouAEIOU"
count = 0
for ch in s:
    if ch in vowels:
        count += 1
print(count)     # Output: 3
```

⑥ Check palindrome string.

```
s = "madam"
if s == s[::-1]:
    print("Palindrome")
else:
    print("Not Palindrome")
# Output: Palindrome
```

⑦ Count frequency of characters using dictionary.

```
s = "programming"
freq = {}
for ch in s:
    freq[ch] = freq.get(ch, 0) + 1
print(freq)   # Output: {'p': 1, 'r': 2, 'o': 1,
      'g': 2, 'a': 1, 'm': 2, 'i': 1, 'n': 1}
```

⑧ Merge two dictionaries.

```
d1 = {'a': 1, 'b': 2}
d2 = {'b': 3, 'c': 4}
d1.update(d2)
print(d1)   # Output: {'a': 1, 'b': 3, 'c': 4}
```

⑨ Word count in a sentence using dictionary.

```
sentence = "this is a simple test this is"
words = sentence.split()
wc = {}
for w in words:
    wc[w] = wc.get(w, 0) + 1
print(wc)   # Output: {'this': 2, 'is': 2,
          'a': 1, 'simple': 1, 'test': 1}
```

⑩ Union and intersection of two sets.

```
s1 = {1, 2, 3, 4}
s2 = {3, 4, 5, 6}
union_set = s1 | s2
inter_set = s1 & s2
print(union_set)    # Output: {1, 2, 3, 4, 5, 6}
print(inter_set)    # Output: {3, 4}
```

⑪ Create squares list using list comprehension.

```
n = 5
squares = [x*x for x in range(1, n+1)]
print(squares)    # Output: [1, 4, 9, 16, 25]
```

⑫ Create dictionary of numbers and squares using dictionary comprehension.

```
n = 5
sq = {x: x*x for x in range(1, n+1)}
print(sq)   # Output: {1: 1, 2: 4, 3: 9,
                       4: 16, 5: 25}
```